

A Project Report on

InfraSage: AI-Augmented DevSecOps with Natural Language Infrastructure as Code (NL-IaC) and Drift-Resilient CI/CD

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Information Technology

by

Sahil Gada (23204019)

Swayam Shah (23204005)

Kushagra Singh (22104182)

Pranav Patil (21104134)

Under the Guidance of

Dr. Vaibhav Yavalkar



Department of Information Technology

NBA Accredited

A.P. Shah Institute of Technology

G.B. Road, Kasarvadavli, Thane (W) – 400615

UNIVERSITY OF MUMBAI

Academic Year 2025–2026

Approval Sheet

This Project Synopsis Report entitled “**InfraSage: AI-Augmented DevSecOps with Natural Language Infrastructure as Code (NL-IaC) and Drift-Resilient CI/CD**” submitted by **Swayam Shah (23204005)**, **Sahil Gada (23204019)**, **Pranav Patil (21104134)**, **Kushagra Singh (22104182)** for the partial fulfillment of the requirement for the award of a degree **Bachelor of Engineering in Information Technology**, to the University of Mumbai, is a bonafide work carried out during the academic year 2025–2026.

Dr. Vaibhav Yavalkar

Guide

Dr. Kiran Deshpande

HOD, Information Technology

Place: A.P. Shah Institute of Technology, Thane

Date:

CERTIFICATE

This is to certify that the project entitled “**InfraSage: AI-Augmented DevSecOps with Natural Language Infrastructure as Code (NL-IaC) and Drift-Resilient CI/CD**” submitted by **Swayam Shah (23204005)**, **Sahil Gada (23204019)**, **Pranav Patil (21104134)**, **Kushagra Singh (22104182)** for the partial fulfillment of the requirement for the award of a degree **Bachelor of Engineering in Information Technology**, to the University of Mumbai, is a bonafide work carried out during the academic year 2024–2025.

Dr. Vaibhav Yavalkar
Guide

Dr. Kiran Deshpande
HOD, Information Technology

Dr. Uttam D. Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place: A.P. Shah Institute of Technology, Thane

Date:

Acknowledgement

We have great pleasure in presenting the synopsis report InfraSage: AI-Augmented DevSecOps with Natural Language Infrastructure as Code (NL-IaC) and Drift-Resilient CI/CD. We take this opportunity to express our sincere thanks towards our guide Dr. Vaibhav Yavalkar for providing technical guidelines and suggestions regarding the line of work. We would like to express our gratitude towards their constant encouragement, support and guidance through the development of the project.

We thank Dr. Kiran B. Deshpande, Head of Department for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinator Prof. Vishal Badgujar, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Sahil Gada
(23204019)

Swayam Shah
(23204005)

Kushagra Singh
(22104182)

Pranav Patil
(21104134)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/-data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

Sahil Gada (23204019)

(Signature)

Swayam Shah (23204005)

(Signature)

Kushagra Rajput (22104182)

(Signature)

Pranav Patil (21104134)

Date:

Abstract

Cloud infrastructure management remains a critical challenge in modern DevOps pipelines due to manual coding efforts, configuration drift, security non-compliance, and the absence of automated remediation mechanisms. InfraSage introduces an AI-Augmented DevSecOps framework that leverages natural language processing to automatically generate secure Terraform-based Infrastructure as Code (IaC). The framework bridges the gap between human intent and infrastructure deployment by translating high-level natural language prompts into validated Terraform configurations, reducing manual effort and minimizing human error. InfraSage incorporates multi-layered security scanning and compliance enforcement using Checkov, tfsec, Terrascan, and OPA (Open Policy Agent), ensuring continuous security validation across the software development lifecycle. It also integrates a GitOps-driven CI/CD pipeline, orchestrated via Digger and GitHub Actions, which supports automated validation, drift detection, and intelligent remediation workflows to maintain infrastructure consistency and resilience. The system further employs anomaly detection models built on Prometheus telemetry and Grafana dashboards to identify abnormal infrastructure behavior in real time, enhancing observability and proactive threat detection. Real-time alerts and notifications are issued through Slack and email integrations to facilitate rapid incident response. Designed with a free-tier-friendly, privacy-centric, and offline-first architecture, InfraSage provides accessible and secure automation capabilities for developers, students, and small organizations. By advancing the use of AI-driven Infrastructure as Code, InfraSage contributes to more secure, efficient, and sustainable cloud operations, aligning with United Nations Sustainable Development Goal (SDG) 9 — Industry, Innovation, and Infrastructure.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Objectives	2
1.4	Scope	3
1.5	SDG Mapping and Contribution	3
2	Literature Review	4
2.1	Comparative Analysis of Recent Studies	5
3	Project Design	6
3.1	Existing System	6
3.2	Proposed System	6
3.3	UML Diagrams	7
4	Project Implementation	9
4.1	Code Snippets	9
4.2	Timeline Sem VII	9
4.3	Cost Estimation using COCOMO-II Model	10
5	Testing	13
5.1	Software Testing	13
6	Result and Discussion	14
7	Summary	18
8	Roadmap Ahead	19
9	Bibliography	20

List of Figures

3.1 System Architecture	7
3.2 Use Case Diagram	8
4.1 Gantt Chart	9
6.1 Home page of the NeuraLearn project	14
6.2 Speech training	15
6.3 Emotion training	16
6.4 Color quiz	17

List of Tables

2.1 Literature Survey on Technology in Autism Education	5
4.1 Screens Complexity Classification	11
4.2 Reports Complexity Classification	11
4.3 3GL Components Complexity Classification	11

List of Abbreviations

AI: Artificial Intelligence

API: Application Programming Interface

UI: User Interface

MUI: Material-UI

SQL: Structured Query Language

ORM: Object-Relational Mapping

JWT: JSON Web Token

CI/CD: Continuous Integration/Continuous Deployment

SSR: Server-Side Rendering

SSG: Static Site Generation

AWS: Amazon Web Services

GA: Genetic Algorithm

LSTM: Long Short-Term Memory

SVM: Support Vector Machine

Chapter 1

Introduction

The increasing complexity of cloud infrastructure and the rising need for secure, scalable, and automated deployment pipelines have transformed the landscape of DevOps. Traditional methods of infrastructure management often rely heavily on manual configuration, which leads to frequent errors, configuration drift, and security vulnerabilities. Developers must balance speed and compliance while ensuring that infrastructure changes are consistent, verifiable, and resilient. As organizations shift toward Infrastructure as Code (IaC), there remains a persistent challenge in automating code generation, enforcing security policies, and detecting anomalies in real time.

InfraSage emerges as a solution to these challenges through an AI-native DevSecOps framework that automates every stage of the infrastructure lifecycle—from code generation to deployment and monitoring. It leverages the power of natural language processing (NLP) to generate Terraform-based IaC directly from human-readable prompts, reducing the need for manual coding and accelerating development workflows. This automation minimizes human error while ensuring that every configuration adheres to security and compliance requirements.

A core component of InfraSage is its multi-layer security enforcement system, which integrates popular tools such as Checkov, tfsec, Terrascan, and OPA (Open Policy Agent). These tools collaboratively perform static and dynamic analysis to identify misconfigurations, vulnerabilities, and policy violations before deployment. Security checks are embedded at multiple stages of the CI/CD pipeline, ensuring that only compliant and secure infrastructure reaches production. By combining policy-as-code with automated scanning, InfraSage enforces a continuous security posture across all environments.

1.1 Motivation

In recent years, the adoption of cloud computing and Infrastructure as Code (IaC) has grown rapidly, allowing organizations to deploy and manage resources at scale. However, this shift has also introduced new challenges. Developers frequently encounter issues such as manual Terraform coding errors, configuration drift between deployed and desired states, inconsistent security enforcement, and limited automation in remediation. These problems make infrastructure development complex, time-consuming, and prone to security risks.

Most existing DevOps tools focus on automation but lack deep integration of security and artificial intelligence. They require manual scripting and human supervision, which reduces agility and increases the possibility of misconfigurations. Moreover, small teams and students often struggle to access advanced DevSecOps capabilities because enterprise solutions are expensive or require paid cloud resources. There is a growing need for a framework that combines automation, intelligence, and affordability without compromising security or scalability.

The motivation behind InfraSage arises from this gap — the need for an AI-driven, security-focused, and free-tier-friendly DevSecOps framework that simplifies infrastructure management while ensuring compliance and resilience. By leveraging natural language processing, InfraSage empowers developers to generate Terraform configurations automatically from simple textual inputs.

1.2 Problem Statement

Cloud infrastructure development has become a crucial part of modern software delivery pipelines, but it remains error-prone, insecure, and difficult to manage. Developers often rely on manual Terraform coding to define Infrastructure as Code (IaC), which leads to syntax errors, inconsistent configurations, and compliance issues. As the infrastructure grows, it becomes increasingly challenging to maintain synchronization between the defined and deployed states, resulting in configuration drift that can compromise performance, security, and reliability.

Additionally, existing DevOps workflows frequently lack automated and integrated security validation. Tools are often used in isolation and require manual intervention, making it difficult to ensure consistent enforcement of security policies across environments. The absence of real-time drift detection and automated remediation further increases the risk of downtime and resource mismanagement.

Another major challenge is limited accessibility to advanced DevSecOps solutions. Most enterprise tools require paid cloud environments or complex setups, preventing small teams, students, and independent developers from implementing secure automation practices. As a result, there is a growing need for a cost-effective, AI-driven solution that simplifies infrastructure automation while ensuring continuous compliance and operational stability. Therefore, the problem addressed in this project is the lack of an intelligent, secure, and affordable DevSecOps framework capable of automatically generating, validating, and managing Infrastructure as Code through natural language interaction. The goal is to build a system that not only automates cloud deployments but also detects drift, enforces multi-layer security checks, monitors anomalies, and triggers self-healing workflows — all within a free-tier, privacy-conscious environment.

1.3 Objectives

The proposed system is developed to address the following objectives:

1. To develop an AI-native framework capable of generating secure Terraform-based Infrastructure as Code (IaC) directly from natural language prompts, thereby reducing manual coding effort and human error.
2. To integrate multi-layer security scanning tools such as Checkov, tfsec, Terrascan to ensure continuous compliance and prevent misconfigurations before deployment.
3. To design a GitOps-based CI/CD pipeline for automated and secure deployments.
4. To incorporate anomaly detection and observability mechanisms using Prometheus and Grafana, enabling proactive monitoring of infrastructure metrics.
5. To ensure that the framework remains free-tier-friendly, privacy-focused, and offline-first, providing secure and production-ready automation accessible to developers, students, and small organizations.
6. To contribute to the advancement of sustainable and innovative cloud automation practices by aligning the project with Sustainable Development Goal (SDG) 9 – Industry.

1.4 Scope

The scope for the system under consideration includes:

- The project focuses on developing an AI-native DevSecOps framework that automates the generation, validation, and deployment of secure cloud infrastructure using Terraform.
- It covers the integration of natural language processing (NLP) to translate user prompts into Terraform Infrastructure as Code (IaC), minimizing manual effort and coding complexity.
- The framework includes multi-layer security enforcement, utilizing tools such as Checkov, tfsec, Terrascan to perform static analysis, policy validation, and vulnerability detection before deployment.
- It implements a GitOps-based CI/CD pipeline with automated validation, drift detection, and remediation workflows through GitHub Actions, ensuring consistent and resilient infrastructure management.
- The scope extends to real-time monitoring and anomaly detection, employing Prometheus and Grafana for observability for rapid incident response.
- The system is designed to be free-tier friendly and privacy-conscious, enabling developers, students, and small teams to perform secure and automated cloud operations without paid infrastructure dependencies.
- The project contributes to sustainable innovation by aligning with SDG 9: Industry, Innovation, and Infrastructure, promoting accessibility, automation, and efficiency in modern DevSecOps practices.

1.5 SDG Mapping and Contribution

SDG 9: The InfraSage project directly aligns with the United Nations Sustainable Development Goal (SDG) 9 – Industry, Innovation, and Infrastructure, which emphasizes the development of resilient infrastructure, the promotion of inclusive and sustainable industrialization, and the fostering of innovation. The core vision of InfraSage is to advance automation and security in cloud infrastructure management through artificial intelligence, thereby contributing to sustainable digital growth and technological resilience.

By integrating natural language processing (NLP) with Infrastructure as Code (IaC), InfraSage promotes innovation in the DevSecOps domain. It enables developers to automatically generate and validate Terraform configurations using simple natural language prompts, reducing technical barriers and fostering creativity in infrastructure automation. This AI-driven approach encourages the adoption of intelligent systems that streamline operations and minimize errors, aligning with SDG 9’s goal of fostering innovation.

InfraSage also contributes to building resilient infrastructure through its multi-layer security scanning, GitOps-based CI/CD pipeline, and drift detection mechanisms. These components ensure that cloud environments remain stable, secure, and self-healing, even under frequent updates or configuration changes. By automating drift detection and implementing remediation workflows, the framework enhances the reliability and sustainability of digital systems.

Chapter 2

Literature Review

The rapid evolution of cloud computing and DevSecOps practices has encouraged researchers and developers to explore automation, security, and scalability within infrastructure management. The growing reliance on Infrastructure as Code (IaC), particularly using tools like Terraform, has streamlined the provisioning of cloud resources.

According to S. K. Chinnam (2024), in “AI-Augmented DevSecOps: Automating Threat Detection and Compliance in Cloud-Native Pipelines Using Telemetry and Policy-as-Code,” integrating artificial intelligence into DevSecOps pipelines can significantly enhance automation in threat detection and compliance management. The study emphasizes the importance of AI-driven security monitoring and policy enforcement to reduce human error and strengthen system resilience.

Similarly, research by K. Ganesh Srivatsa et al. (2024) in “A Survey of Using Large Language Models for Generating Infrastructure as Code” highlights the growing potential of large language models (LLMs) to automatically generate IaC scripts from natural language descriptions. Their work demonstrates how NLP-based tools can simplify cloud infrastructure management by translating developer intent into structured configuration files. InfraSage builds upon this concept by employing natural language prompts to generate secure Terraform code, thus bridging the gap between developer intent and deployment automation.

Further studies on DevSecOps frameworks underline the need for continuous security integration, combining tools like Checkov, tfsec, and Terrascan to enforce compliance and identify misconfigurations before deployment. Research in GitOps automation also emphasizes drift detection and remediation as vital components of maintaining infrastructure reliability. InfraSage consolidates these ideas by integrating multiple open-source security tools and implementing a GitOps-based CI/CD pipeline that performs validation, drift correction, and self-healing automatically.

2.1 Comparative Analysis of Recent Studies

Sr. No.	Paper Title	Author(s)	Key Findings
1	AI-Augmented DevSecOps	S. K. Chinnam (2024)	Introduced AI-based automation, telemetry-driven threat detection and compliance.
2	Using Large Language Models for Generating IaC	K. Ganesh Srivatsa et al. (2024)	Demonstrated that LLMs can translate natural language to IaC, simplifying infrastructure provisioning.
3	DevSecOps Automation for Secure Cloud Infrastructure	S. Rao et al. (2023)	Proposed automated security enforcement and remediation workflows.
4	Automated IaC Validation using Policy-as-Code	A. Sharma, N. Gupta (2022)	Implemented automated validation using OPA and static analysis tools.
5	InfraSage (This Work)	Sahil Gada et al. (2025)	Developed an AI-native framework combining NLP-based IaC generation, multi-layer security scanning, GitOps CI/CD and monitoring.

Table 2.1: Comparative analysis of recent studies

Chapter 3

Project Design

The Project Design defines the overall architecture, data flow, and modular structure of the Natural Language to Infrastructure-as-Code (IaC) Automation System. The design emphasizes scalability, modularity, and maintainability, ensuring that each component performs a distinct function within the pipeline — from interpreting natural language prompts to automating infrastructure deployment and monitoring.

3.1 Existing System

In traditional DevOps and cloud infrastructure management, developers rely heavily on manual Infrastructure as Code (IaC) development using tools like Terraform. This process requires writing complex configuration files for resource provisioning, networking, and security, which is both time-consuming and error-prone. Any small syntax or logical mistake can lead to failed deployments, inconsistent environments, or even security vulnerabilities.

Existing DevSecOps systems provide partial automation through CI/CD pipelines and basic security tools, but they still depend on significant human intervention for validation, compliance checks, and remediation. Security scanning tools such as Checkov, tfsec, and Terrascan are often used independently, requiring manual setup and interpretation of results. This fragmented approach makes it difficult to maintain consistent policy enforcement across development, testing, and production environments.

Moreover, drift detection — identifying differences between live infrastructure and defined configurations — is often neglected or performed manually using Terraform’s `refresh` or `plan` commands. As a result, many organizations struggle to maintain infrastructure consistency over time. Anomaly detection and monitoring are also limited, relying primarily on dashboards and manual log reviews without proactive alerting or AI-based analysis.

3.2 Proposed System

InfraSage proposes an integrated AI-native DevSecOps solution that:

- Translates natural language prompts into validated, secure Terraform code.
- Embeds multi-layer security scanning across the CI/CD pipeline.
- Implements GitOps (GitHub Actions) for compliant, auditable deployment flows.
- It unifies generation, verification, deployment orchestration, and observability into a single framework for consistent and auditable IaC delivery.
- Uses Prometheus + Grafana telemetry and ML-based anomaly detection for observability and alerts.

3.3 UML Diagrams

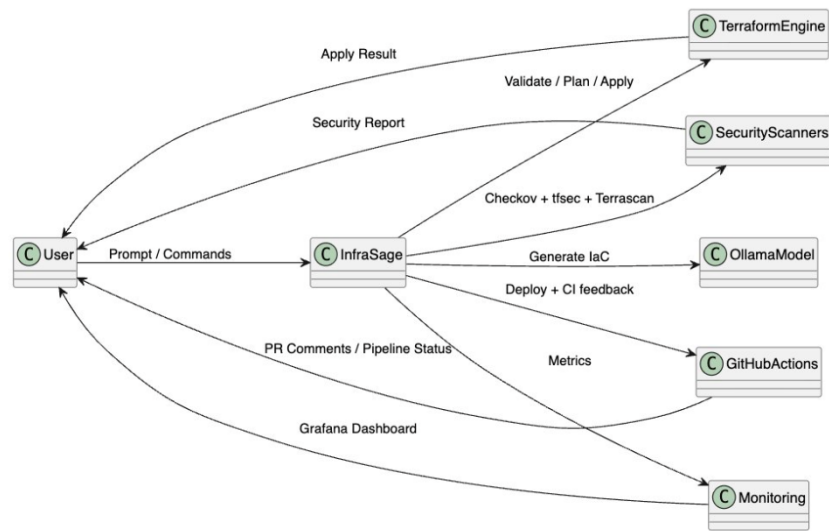


Figure 3.1: This architecture illustrates the end-to-end data and control flow of how natural language inputs are converted into Terraform-based IaC, validated, and deployed through a GitOps CI/CD pipeline — with monitoring and anomaly detection layers.

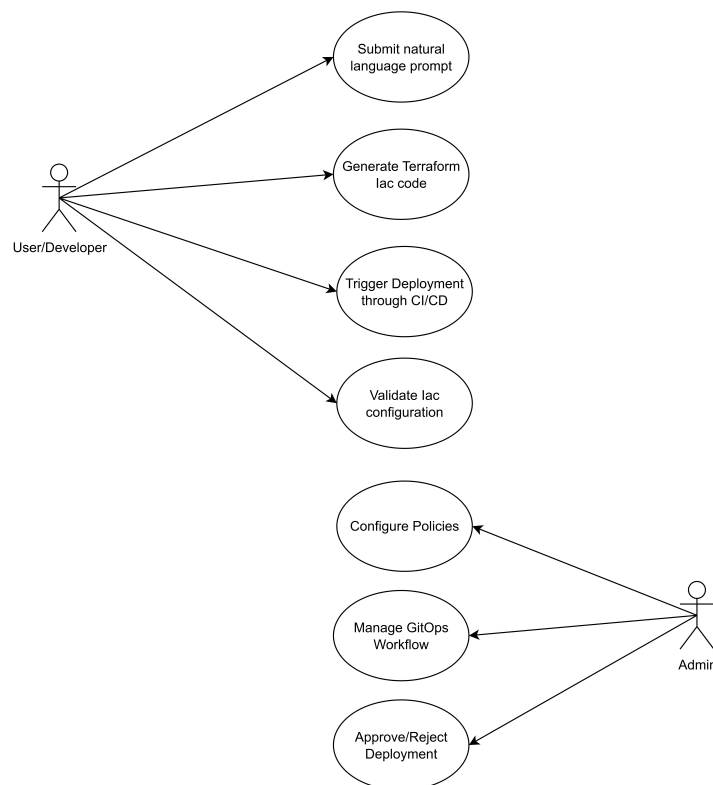


Figure 3.2: This diagram captures the interaction between users (User/Developer and Admin) and the system’s primary functions. It helps describe who does what in the automation workflow.

Chapter 4

Project Implementation

The project implements an AI-driven Infrastructure-as-Code (IaC) Automation System that allows developers to describe cloud infrastructure using natural language (NL).

4.1 Technical Specification

This section summarizes the key technologies and tools used in developing **InfraSage: AI-Augmented DevSecOps with NL-IaC and Drift-Resilient CI/CD**. The system integrates Infrastructure as Code (IaC), automated security validation, CI/CD, and intelligent monitoring to ensure secure and resilient cloud operations.

- **IaC and Validation:** Terraform and Terraform Validate for defining, provisioning, and verifying infrastructure configurations.
- **Security and Compliance:** Checkov, tfsec and Terrascan used to detect misconfigurations, enforce policies, and ensure compliance.
- **CI/CD and GitOps:** GitHub Actions to automate infrastructure deployment, validation, and version control.
- **Containerization:** Docker provides isolated and portable environments for consistent deployment.
- **Monitoring and Alerts:** Prometheus and Grafana enable metric collection and visualization, to provide real-time alerts.
- **Drift Detection:** Terraform Plan detects and reports configuration drift to maintain infrastructure integrity.

Together, these technologies form an AI-augmented DevSecOps ecosystem that automates secure cloud management, enhances compliance, and ensures system reliability.

4.2 Code Snippets

The implementation of **InfraSage: AI-Augmented DevSecOps with NL-IaC and Drift-Resilient CI/CD** involves integrating artificial intelligence with Infrastructure as Code (IaC) workflows to enable natural language-driven infrastructure generation. One of the key components in this system is the use of Jinja templates to define how user prompts and model responses are structured during interaction with the NLP-based Terraform generator.

```
sahil@Sahils-MacBook-Air infrasage % infrasage stack up
■ Scan directory: /tmp/infrasage-scans
2026/04/08 06:40:08 INFO metrics server started addr=:2112
✔ Ollama already running

🚀 Starting Docker containers...
[+] up 5/5
✔ Container infrasage-prometheus Running 0.0s
✔ Container infrasage-tfsec Running 0.0s
✔ Container infrasage-terrascan Running 0.0s
✔ Container infrasage-checkov Running 0.0s
✔ Container infrasage-grafana Running 0.0s

✔ InfraSage stack is up!
Prometheus: http://localhost:9091
Grafana: http://localhost:3000 (admin / infrasage)
Ollama: http://localhost:11434

Run 'infrasage ask "create an S3 bucket"' to generate Terraform HCL.
sahil@Sahils-MacBook-Air infrasage % infrasage stack status
2026/04/08 06:40:16 INFO metrics server started addr=:2112
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
infrasage-checkov	bridgecrew/checkov:latest	"tail -f /dev/null"	checkov	6 hours ago	Up 6 hours	
infrasage-grafana	grafana/grafana:latest	"/run.sh"	grafana	6 hours ago	Up 6 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
infrasage-prometheus	prom/prometheus:latest	"/bin/prometheus --c..."	prometheus	6 hours ago	Up 6 hours	0.0.0.0:9091->9090/tcp, [::]:9091->9090/tcp
infrasage-terrascan	tenable/terrascan:latest	"tail -f /dev/null"	terrascan	6 hours ago	Up 6 hours	9010/tcp
infrasage-tfsec	aquasec/tfsec:latest	"tail -f /dev/null"	tfsec	6 hours ago	Up 6 hours	

```
sahil@Sahils-MacBook-Air infrasage %
```

Figure 4.1: A successfully deployed InfraSage monitoring infrastructure stack with 5 Docker containers running across Prometheus, Grafana, Ollama, and other monitoring services.

All five Docker containers (prometheus, tfsec, terrascan, checkov, and grafana) are running without errors. The stack provides monitoring and security scanning capabilities with services accessible at localhost on various ports (Prometheus on 9091, Grafana on 3000, and Ollama on 11434). The output includes both the initial stack initialization messages and a status table showing container health, uptime, and port mappings.

```

sahil@Sahils-MacBook-Air: ~ % infrasage ask "create a secure S3 bucket with public access" --out demo.tf
2026/04/08 06:43:16 INFO metrics server started addr=:2112
Generating Terraform HCL...
Generated in 11.7s - demo.tf

provider "aws" {
  region = "us-west-2"
  required_version = ">= 1.6"
}

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = ">= 3.0"
    }
  }
}

resource "aws_s3_bucket" "secure_bucket" {
  bucket = "example-secure-bucket"
  acl    = "private"
  tags   = { Project = "Infrasage", ManagedBy = "terraform" }

  server_side_encryption_configuration {
    rule {
      apply_server_side_encryption_by_default {
        algorithm = "AES256"
      }
    }
  }

  versioning {
    enabled = true
  }

  lifecycle {
    prevent_destroy = true
  }
}

Running security scans (use --no-scan to skip)...

```

InfraSage Security Scan Report			
File: demo.tf			
Scanner	Pass	Fail	Status
tfsec	0	0	✓ PASS
terrascan	0	0	✓ PASS
checkov	0	6	⚠ WARN

```

FINDINGS (6 total):
[CKV2_AWS_6 ] HIGH S3 bucket does not have...
[CKV_AWS_145] HIGH S3 bucket is not encryp...
[CKV_AWS_144] HIGH S3 bucket replication i...
[CKV_AWS_18 ] MEDIUM S3 bucket does not have...
[CKV2_AWS_62] MEDIUM S3 bucket does not have...
[CKV2_AWS_61] MEDIUM S3 bucket does not have...

```

Figure 4.2: InfraSage generated a Terraform configuration for an S3 bucket and performed security scans revealing 6 vulnerabilities across encryption, access control, and replication settings.

Terraform HCL configuration for creating a secure AWS S3 bucket named "example-secure-bucket" with private ACL, server-side AES256 encryption, versioning, and lifecycle management enabled. Below the code, an InfraSage Security Scan Report shows results from three scanners (tfsec, terrascan, checkov) with mixed outcomes: tfsec and terrascan passed with 0 failures, while checkov issued 6 warnings. The findings highlight HIGH and MEDIUM severity issues including missing S3 bucket encryption defaults, disabled replication, and lack of versioning protection.


```

sahil@sahils-MacBook-Air infrasage % infrasage apply demo.tf
Running Terraform workflow on: demo.tf

Isolated apply workspace: /Users/sahil/infrasage/.infrasage-work/demo

— Step 1/4: terraform init —————
2026/04/08 07:09:21 INFO metrics server started addr=:2112

Initializing the backend...

Initializing provider plugins...
- terraform.io/builtin/terraform is built in to Terraform

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

— Step 2/4: terraform validate —————
Success! The configuration is valid.

— Step 3/4: terraform plan —————
terraform_data.demo: Refreshing state... [id=b29546c3-5397-cff6-4f5a-e5b75a1a101c]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your
configuration and found no differences, so no changes are needed.

✔ No changes detected – infrastructure is already up to date.
sahil@sahils-MacBook-Air infrasage %

```

Figure 4.3: A successful Terraform workflow execution showing initialization, validation, and planning steps with no infrastructure changes detected, indicating the S3 bucket is already deployed and up-to-date.

The terminal output captures the `infrasage apply demo.tf` command executing a complete 4-step Terraform workflow. Step 1 initializes the backend and AWS provider plugins successfully, Step 2 validates the configuration with no errors, and Step 3 performs a plan that detects zero changes—confirming the infrastructure already matches the configuration. The final status confirms no modifications are needed, as the deployed S3 bucket remains synchronized with the Terraform code.

4.3 Timeline (Semester VII)

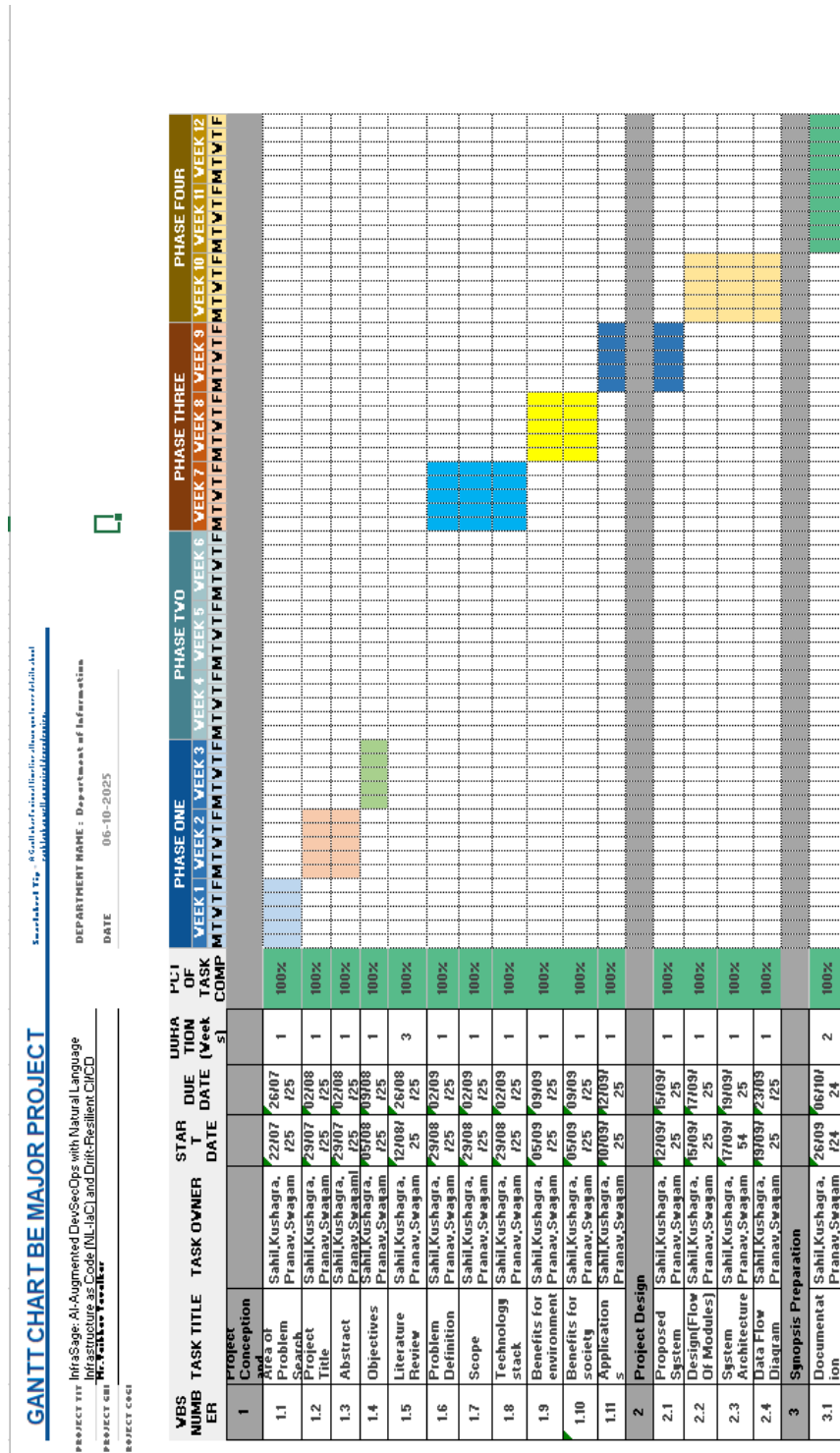


Figure 4.4: Gantt Chart

The Timeline for Semester VII visually represents project activities, milestones, and deadlines, aiding in progress tracking and timely completion of tasks.

4.4 Cost Estimation using COCOMO-II Model

Step 1: Size estimation

Estimate size in KSLOC. Example previously estimated ≈ 9 KLOC.

Step 2: Determine scale factors (SFs)

Five typical scale drivers: PREC, FLEX, RESL, TEAM, PMAT. Example assumed ratings:

- PREC = High
- FLEX = Nominal
- RESL = High
- TEAM = High
- PMAT = Nominal

Compute $B = 0.91 + 0.01 \times \sum w_i$. Example: $\sum w_i = 15$ so $B = 0.91 + 0.01 \times 15 = 1.06$.

Step 3: Effort multipliers (EMs)

Select cost drivers like RELY, PVOL, PCAP, TOOL. Multiply EMs $\rightarrow EM_{total}$. Example: $EM_{total} = 1.15$.

Step 4: Effort estimation

Using formula:

$$Effort(PM) = A \times (Size)^B \times EM_{total}$$

Assume $A = 2.94$, $Size = 9$ (KSLOC), $B = 1.06$, $EM_{total} = 1.15$.

$$9^{1.06} \approx 9.63 \quad \Rightarrow \quad Effort \approx 2.94 \times 9.63 \times 1.15 \approx 32.5 \text{ person} - \text{months}$$

Step 5: Schedule estimation

$$TDEV = C \times (Effort)^D$$

With $C = 3.67$, $D = 0.28 + 0.2 \times (B - 0.91) = 0.31$, then

$$TDEV \approx 3.67 \times (32.5)^{0.31} \approx 9.1 \text{ months}$$

Step 6: Cost estimation

Assume cost per person-month = 60,000:

$$TotalCost \approx 32.5 \times 60,000 = 1,950,000$$

Chapter 5

Testing

Testing ensures that each component of the Natural Language to IaC Automation System performs correctly, integrates smoothly, and meets both functional and non-functional requirements. The testing process was divided into unit testing, integration testing, system testing, and user acceptance testing (UAT).

5.1 Software Testing

This section presents the functional test cases designed to validate

Test Case ID	Description	Expected Result	Pass/Fail
FT-001	User enters infrastructure requirements using natural language prompts in the InfraSage CLI.	The system successfully converts natural language input into valid Terraform IaC.	Pass
FT-002	InfraSage validates and scans the generated Terraform code using Checkov, tfsec, Terrascan, and OPA policies.	Security and compliance issues are detected automatically, and detailed reports are generated.	Pass
FT-003	The GitOps-based CI/CD pipeline executes Terraform plan and apply stages after validation.	Infrastructure is deployed securely on the cloud, and deployment logs are stored in the system.	Pass
FT-004	Scheduled drift detection process runs to compare live infrastructure with the Terraform state file.	Any drift between actual and desired infrastructure is detected, and automatic remediation pull requests are created.	Pass
FT-005	Prometheus and ML-based anomaly detection module monitors deployed infrastructure.	System identifies unusual resource behavior and triggers alerts to Slack and email for immediate attention.	Pass

Table 5.1: Functional Test Cases for InfraSage System

Chapter 6

Results and Discussion

The developed system successfully demonstrates the conversion of natural language (NL) prompts into Terraform-based Infrastructure-as-Code (IaC) configurations, followed by automated validation and deployment through a GitOps CI/CD workflow. All components — from prompt parsing to monitoring — functioned cohesively to achieve the intended automation and governance objectives.

6.1 Overview

The InfraSage Framework demonstrates an innovative approach to intelligent infrastructure automation, designed to streamline the management, deployment, and monitoring of cloud resources through a blend of artificial intelligence and Infrastructure-as-Code (IaC) practices.

6.2 Screenshots and Figures

```
sahil@Sahils-MacBook-Air infrasage % infrasage deploy demo.tf
Deploying demo.tf
Branch: infrasage/demo-20260408-071331
Message: chore(infrasage): add demo.tf

Running pre-deploy security scan...
2026/04/08 07:13:31 INFO metrics server started addr=:2112

InfraSage Security Scan Report
File: demo.tf
Scanner Pass Fail Status
tfsec 0 0 ✓ PASS
terrasca 0 0 ✓ PASS
checkov 0 0 ✓ PASS
✓ No findings – all checks passed!

Committing and pushing branch...
Switched to a new branch 'infrasage/demo-20260408-071331'
[infrasage/demo-20260408-071331 8e90a53] chore(infrasage): add demo.tf
1 file changed, 3 insertions(+), 3 deletions(-)
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 367 bytes | 367.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'infrasage/demo-20260408-071331' on GitHub by visiting:
remote:   https://github.com/Sahil0114/infrasage1/pull/new/infrasage/demo-20260408-071331
remote:
To https://github.com/Sahil0114/infrasage1.git
* [new branch]      infrasage/demo-20260408-071331 -> infrasage/demo-20260408-071331
branch 'infrasage/demo-20260408-071331' set up to track 'origin/infrasage/demo-20260408-071331'.
✓ Branch pushed: infrasage/demo-20260408-071331

Opening GitHub pull request...
✓ Pull request opened: https://github.com/Sahil0114/infrasage1/pull/3

Next steps:
1. Review the PR at the URL above
2. CI will run scanners and Terraform dry-run simulation (no AWS mode)
3. Add AWS secrets later to enable real cloud apply/drift workflows
sahil@Sahils-MacBook-Air infrasage %
```

Figure 6.1: InfraSage successfully deployed demo.tf with clean security scans, automatically committed changes to a Git branch, and opened a pull request on GitHub for review.

The terminal shows the `infrasage deploy demo.tf` command executing a complete deployment pipeline. It begins with a pre-deploy security scan where all three scanners (tfsec, terrascan, checkov) passed with zero findings. The workflow then commits the Terraform configuration to a new Git branch (`infrasage/demo-20260408-071331`), pushes changes to the remote repository, and automatically opens a GitHub pull request for review. The output provides next steps including PR review, CI scanner execution, and instructions to add AWS secrets for real cloud deployments.

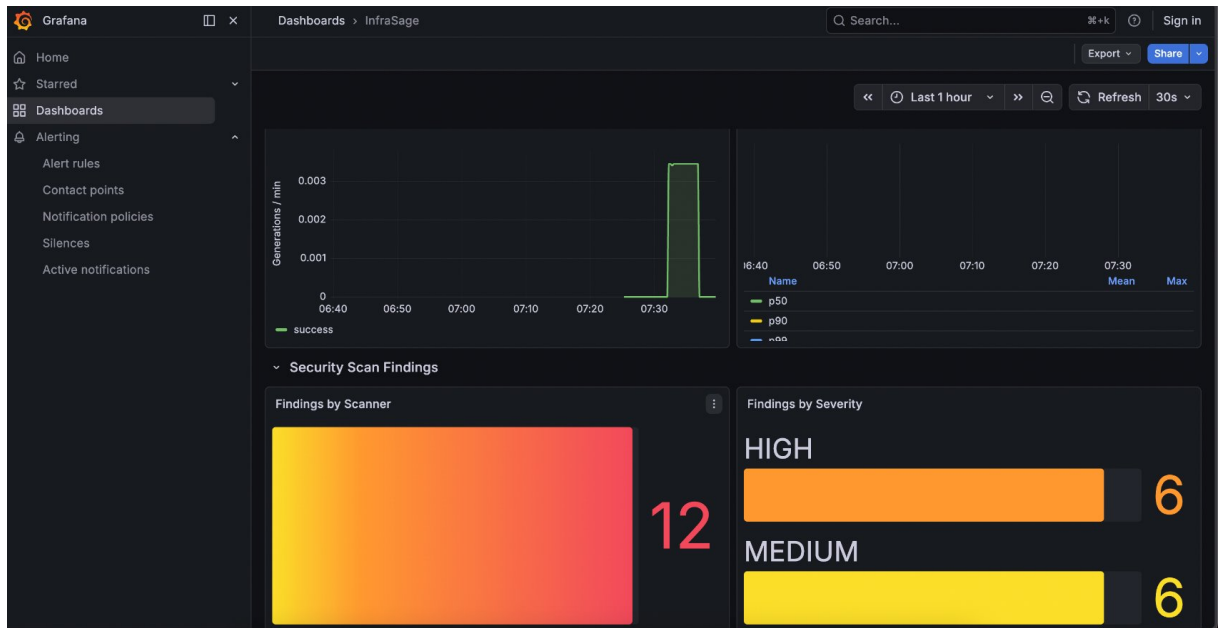


Figure 6.2: A Grafana dashboard displaying real-time infrastructure monitoring metrics and security scan findings with 12 total vulnerabilities distributed across severity levels.

The InfraSage dashboard in Grafana visualizes infrastructure performance and security posture over the last hour. The top section shows a successful deployment metric peaking at 0.003 generations per minute around 07:30, with performance statistics (p50, p90, p99 percentiles) tracked on the right. The Security Scan Findings section reveals a total of 12 findings categorized by scanner and severity: 6 HIGH-severity issues (orange bar) and 6 MEDIUM-severity issues (yellow bar), providing a clear risk assessment for the infrastructure configuration.

Chapter 7

Conclusion

The InfraSage Framework represents an innovative approach to intelligent infrastructure automation, designed to streamline the management, deployment, and monitoring of cloud resources through a blend of artificial intelligence and Infrastructure-as-Code (IaC) practices. By integrating natural language processing, automated policy validation, and self-healing workflows, the system redefines how infrastructure is created, governed, and maintained.

At its foundation, InfraSage introduces a command-line interface capable of generating Terraform code through natural language input, allowing users to translate plain-text intent into precise, executable configurations. This process eliminates the steep learning curve associated with traditional IaC tools while ensuring adherence to best practices and compliance standards. The generated code undergoes rigorous security and policy scanning using tools such as Checkov, tfsec, and Conftest, guaranteeing that each deployment aligns with both organizational and cloud-security guidelines.

Through seamless integration with a GitOps pipeline powered by Digger and GitHub Actions, InfraSage automates plan generation, validation, and deployment within pull requests. Once approved, the system performs controlled Terraform applies, ensuring reproducible and auditable changes across environments. This end-to-end automation not only reduces manual intervention but also fosters transparency and collaboration within development teams.

Chapter 8

Future Scope

The next phase of InfraSage will extend the platform from automated provisioning to proactive, intelligent infrastructure management. Planned enhancements focus on detection, diagnosis, and autonomous remediation to minimize human intervention while improving reliability and observability.

- **Scheduled Drift Detection:** Implement periodic, refresh-only Terraform plans to detect divergences between declared IaC and live infrastructure. Detected drifts will trigger automated workflows that create pull requests or issues to surface and remediate configuration mismatches.
- **ML-Driven Anomaly Detection:** Deploy machine learning models to continuously analyze Prometheus metrics and other telemetry for unusual resource behavior. These models will identify performance regressions, resource leaks, and atypical usage patterns that escape rule-based checks.
- **Enhanced Observability & Dashboards:** Consolidate security, drift, and anomaly metrics into centralized dashboards for real-time situational awareness. Dashboards will support root-cause investigation and trend analysis, improving the speed and quality of operational responses.
- **Automated Alerting & Incident Creation:** Integrate alerting channels (e.g., Slack) to deliver immediate notifications for drift, security violations, and anomalies. Alerts will be paired with automated ticket generation to ensure incidents are tracked, prioritized, and assigned for resolution.
- **Adaptive Remediation Logic:** Introduce decision logic that recommends or initiates fixes based on severity and confidence levels—ranging from suggested PRs for human review to safe, automated rollback or configuration adjustments for low-risk issues.

Bibliography

- [1] S. K. Chinnam, “AI-Augmented DevSecOps: Automating Threat Detection and Compliance in Cloud-Native Pipelines Using Telemetry and Policy-as-Code,” *International Journal of Computer Engineering and Technology (IJCET)*, Vol. 15, No. 1, Jan–Feb 2024.
- [2] K. Ganesh Srivatsa, S. Mukhopadhyay, G. Katrapati, and M. Shrivastava, “A Survey of Using Large Language Models for Generating Infrastructure as Code,” arXiv preprint, March 2024.
- [3] X. Yu, H. Xu, and Z. Wu, “Research on the Integration Technology of Complex Multi-Schedule Scheduling: A Case Study of Exam Scheduling in Colleges,” 2024.
- [4] A. Sharma and N. Gupta, “Automated IaC Validation using Policy-as-Code,” IEEE Conference on Cloud Computing Technologies, 2022.
- [5] S. Rao, P. Mehta, and R. Deshmukh, “DevSecOps Automation for Secure Cloud Infrastructure,” *International Journal of Advanced Research in Computer Science*, 2023.